

Project Report: Multi-Tenant Visitor Management System (VMS) SaaS Platform

1. Executive Summary

The Visitor Management System (VMS) is a modern, cloud-native Software as a Service (SaaS) application designed to handle visitor check-ins, employee host approvals, and security tracking for multiple independent organizations on a single platform. By leveraging a strict multi-tenant architecture, the VMS allows businesses to securely isolate their data while enjoying a unified, premium user interface. The platform replaces traditional paper logbooks with digital QR-code passes, real-time dashboards, and automated email notifications.

2. Introduction

2.1 Purpose

The purpose of this project is to build a scalable and secure digital visitor management solution. In the modern corporate environment, tracking who is inside a building is critical for both security and compliance. This VMS digitizes the entire process.

2.2 Scope

The scope includes a master admin portal, tenant (company) registration, visitor self check-in kiosks, employee portals for managing appointments, receptionist dashboards for walk-ins, and a security scanner interface.

2.3 Objectives

- Digitize visitor logs.
- Provide real-time analytics to building administrators.
- Ensure strict multi-tenant data isolation.
- Generate secure digital entry passes with QR codes.

3. Technology Stack

3.1 Frontend (Client-Side)

- **Framework:** React 18
- **Build Tool:** Vite 5
- **Styling:** Vanilla CSS 3 with Glassmorphism UI patterns
- **Routing:** React Router DOM
- **Icons:** Lucide React
- **QR Generation:** qrcode.react
- **Camera/Biometrics:** react-webcam

3.2 Backend (Server-Side)

- **Runtime:** Node.js 20
- **Framework:** Express.js
- **Authentication:** JSON Web Tokens (JWT) & bcryptjs
- **File Parsing:** multer (for potential uploads)

3.3 Database & Infrastructure

- **Database:** Firebase Admin SDK (Firestore - NoSQL Document Database)
- **Backend Hosting:** Render.com (Web Service)
- **Frontend Hosting:** Vercel
- **Secrets Management:** Render `/etc/secrets` secure injection.

4. System Architecture

4.1 High-Level Design

The VMS follows a decoupled Client-Server architecture. The React frontend interacts with the Express backend via RESTful HTTP APIs. All endpoints (except public registration) are secured using a JWT Bearer token strategy.

4.2 Multi-Tenant Architecture Pattern

The platform implements a **"Single Instance, Shared Database"** multi-tenancy model.

Every document in the database (Users, Visitors, Rooms) is tagged with an `orgId`. A global backend middleware (`rbac.middleware.js`) intercepts all requests and automatically injects the authenticated user's `orgId` into database queries. This guarantees that users from "Company A" can never query or view records belonging to "Company B".

5. Database Schema & Data Models

The system utilizes Firebase Firestore. The collections are modeled as follows:

5.1 Organizations Collection

- `id`: string (UUID)
- `name`: string
- `createdAt`: ISO Date String
- `isActive`: boolean

5.2 Users Collection (Staff/Employees)

- `id`: string
- `orgId`: string (Foreign Key to Organizations)

- email: string
- displayName: string
- role: enum ('admin', 'employee', 'receptionist', 'security')
- department: string
- passwordHash: string

5.3 Visitors Collection

- id: string
- orgId: string
- fullName: string
- phone: string
- email: string
- company: string
- purpose: string
- hostEmployeeId: string
- status: enum ('Pending', 'Approved', 'CheckedIn', 'CheckedOut', 'Rejected')
- qrCodeData: string
- photoBase64: string

5.4 Action Logs Collection

- id: string
- orgId: string
- userId: string (optional)
- visitorId: string (optional)
- action: string (e.g., 'CHECK_IN', 'APPROVE_VISITOR')
- timestamp: ISO Date String

6. System Modules & Features

6.1 Master Admin Portal

The Admin portal is the command center for a company. Administrators can register new employees, assign roles (Receptionist, Security, Host), monitor live building capacity, and view detailed audit logs of all actions within their organization.

6.2 Host Employee Portal

Employees log in to view their scheduled visitors. They have the authority to "Approve" or "Reject" pending visitor requests. Approving a visitor automatically generates a digital pass.

6.3 Receptionist Portal

Receptionists handle walk-in visitors. They can bypass the host approval queue for VIPs, register visitors on the spot, and monitor the live queue of expected guests.

6.4 Security Guard Portal

Designed for speed, the security portal features a built-in QR Code scanner (or manual ID entry). Security guards scan a visitor's digital pass at the gate. The system instantly verifies if the pass is valid, approved, and belongs to the correct date.

6.5 Visitor Pre-Registration

A public-facing link where guests can enter their details and snap a live webcam photo before they arrive at the building.

7. Detailed User Flows

7.1 SaaS Onboarding Flow

1. A new company visits the landing page and clicks "Register Organization".
2. They input their company name and master admin details.
3. The backend creates an `orgId`, creates the admin user, and returns a JWT.
4. The admin is redirected to the dashboard to begin adding staff.

7.2 Visitor Check-In Lifecycle

1. **Pre-Registration:** Visitor fills out the public form (Name, Phone, Host).
2. **Pending State:** Visitor is saved to DB with `status: 'Pending'`.
3. **Host Approval:** Host Employee sees the request on their dashboard and clicks "Approve".
4. **Pass Generation:** Backend generates a secure string, updates status to `Approved`. Visitor can now download their QR pass.
5. **Physical Arrival:** Visitor arrives at the gate and shows the QR code.
6. **Scan & Verify:** Security guard scans the code. System marks visitor as `CheckedIn`.
7. **Departure:** Guard scans code again upon exit. Status becomes `CheckedOut`.

8. Deployment Strategy

8.1 Backend Deployment (Render)

The Node.js server is deployed to Render. Due to security best practices, the Firebase JSON credential file is NEVER committed to version control. Instead, it is uploaded directly to Render's `/etc/secrets` directory. The `db.js` file dynamically resolves this path in production.

8.2 Frontend Deployment (Vercel)

The Vite React application is deployed to Vercel. During the build process, the `VITE_API_URL` environment variable is baked into the static assets, securely linking the frontend to the Render backend URL (e.g., `https://vms-backend.onrender.com/api`).

9. Conclusion

The Multi-Tenant Visitor Management System successfully modernizes building security and administrative workflows. By abstracting the core logic into a reusable multi-tenant architecture, the platform is highly scalable and ready for commercial SaaS distribution. Future enhancements may include integration with Slack/Teams for host notifications and Apple Wallet integration for visitor passes.